

Matrix factorizations

Vladislav Goncharenko

Recap

Lecture 1-2: Intro to RecSys

1. Examples of RecSys
2. Formal problem statement
3. Feedback types
 - a. Explicit
 - b. Implicit
4. Architecture of contemporary RecSys
5. Collaborative Filtering
 - a. User2User
 - b. Item2Item
6. Problems
 - a. Cold start
 - b. Feedback loop



Types of user representations

In general, we want to recommend items (music/movie/article/product) to the user so that the user is happy (we work on long-term metrics)

A typical simplification of a user's happiness is whether a recommendation is liked at the moment.
Or even a bigger simplification —do you like the item as a whole or any item rating for the user

The user himself can also be represented in different ways

1. As an unordered set of items that the user interacted with
2. As an unordered set of items with ratings
3. As a sequence of items with ratings
4. As a sequence of interactions taking into account the context

Revise

girafe
ai

01



Evaluation matrix

Basically, we will represent the user as an unordered set of items with ratings and try to predict the user's rating on other items

Users

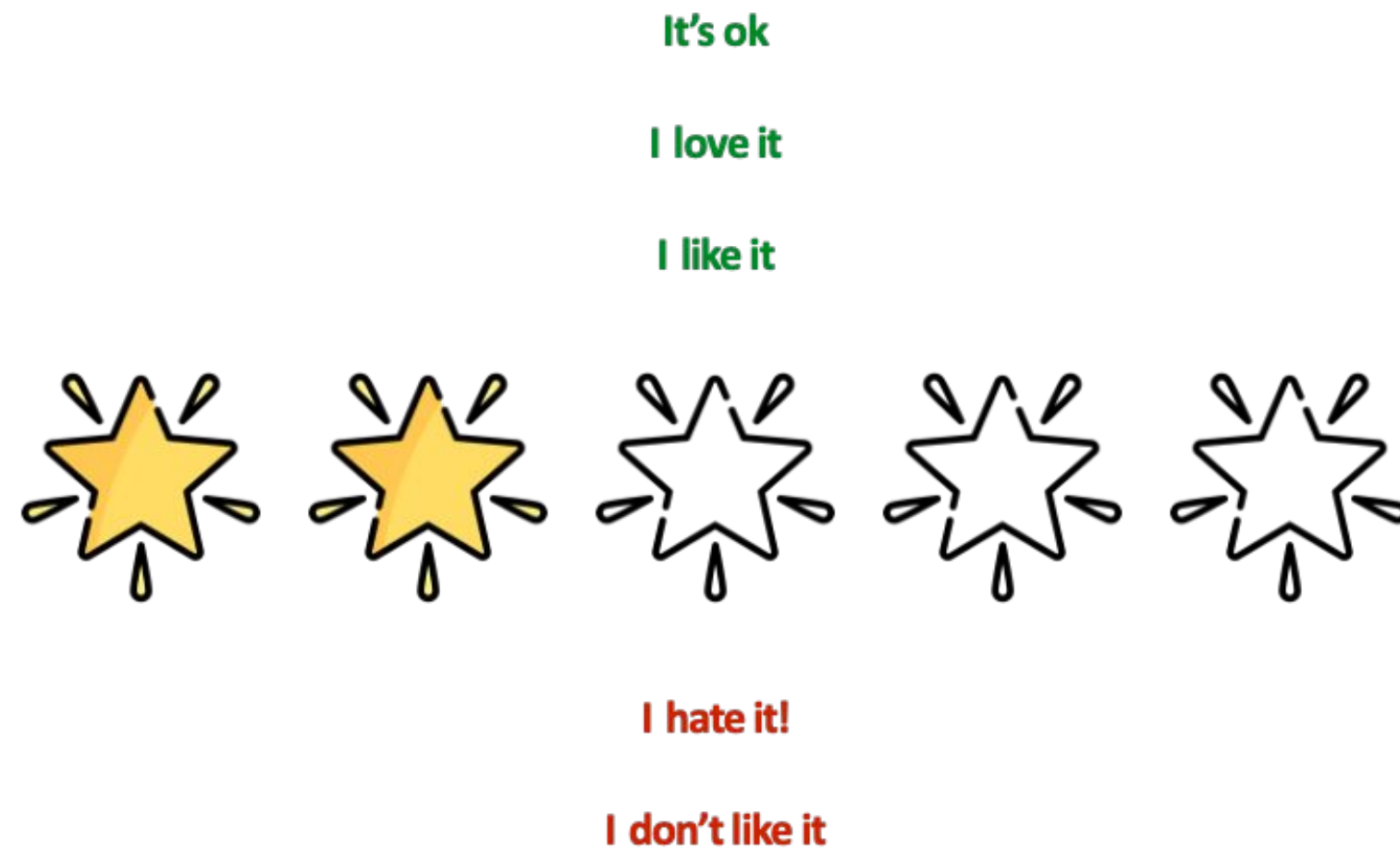


Movie

					
2		2	4	5	
5		4			1
		5		2	
	1		5		4
		4			2
4	5		1		



Types of ratings (feedback)



Explicit (from 1 to 5)



Implicit (have you watched this movie)

Item- and User-based approaches

girafe
ai

02



Idea

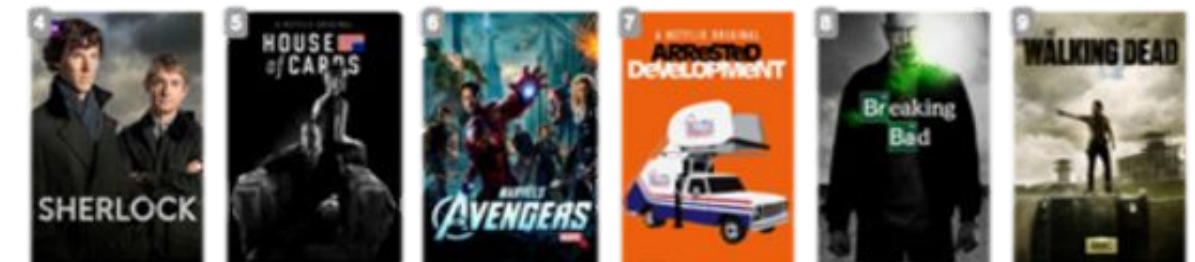
User2User



2		2	4	5	
5		4			1
		5		2	
	1		5		4
		4			2
4	5		1		



Item2Item



2		2	4	5	
5		4			1
		5		2	
	1		5		4
		4			2
4	5		1		



A few formulas

$$\hat{r}_{ui} = \frac{\sum_j s(i, j)(r_{uj} - r_u)}{\sum_j |s(i, j)|} + r_u$$

Explicit:

$$\text{corr}(i, j) = \frac{\sum_u (r_{ui} - r_i)(r_{uj} - r_j)}{\sqrt{\sum_u (r_{ui} - r_i)^2 \sum_u (r_{uj} - r_j)^2}}$$

$s(i, j)$



Implicit:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$



Choosing between Item2Item and User2User

It strongly depends on the number of ratings per user and per item – the more, the more reliable the similarity.

This also leads to the possibility of pre-calculation: if there are a lot of estimates, then adding a pair of estimates will not change anything. Then you can update once in a while.

The similarities themselves can be used for other tasks (candidate generation, contextual recommendations).

A bit of linear algebra

girafe
ai

03



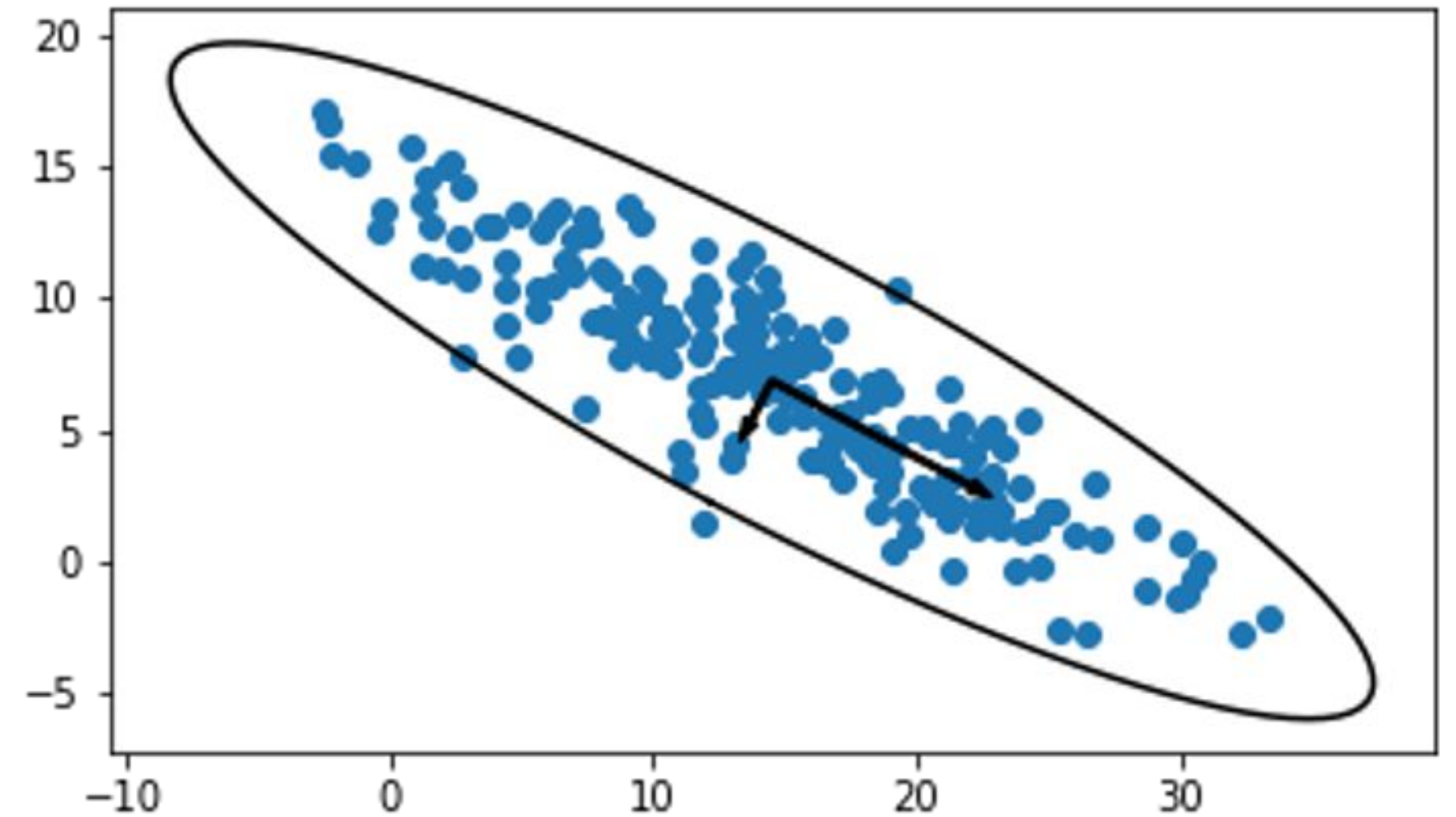
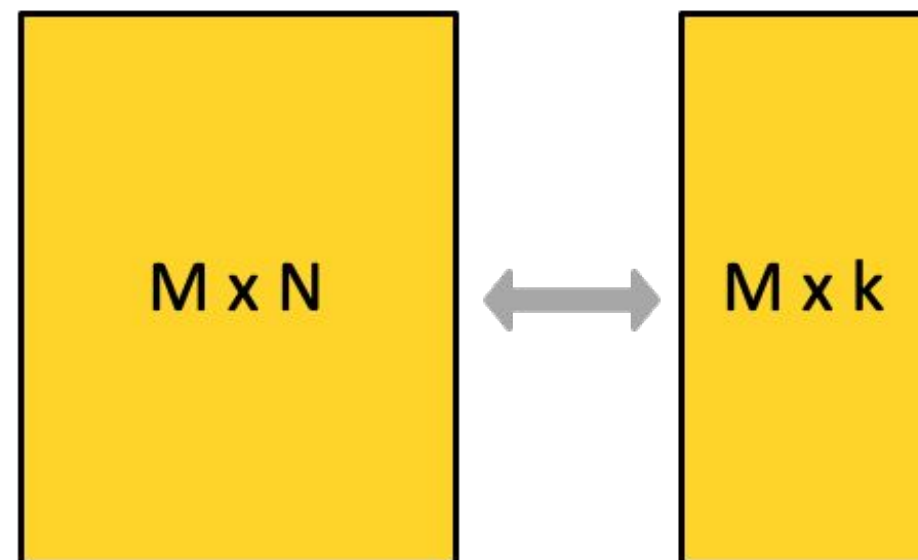
Recall the main things about

- PCA
- SVD
- Truncated SVD



PCA – staging

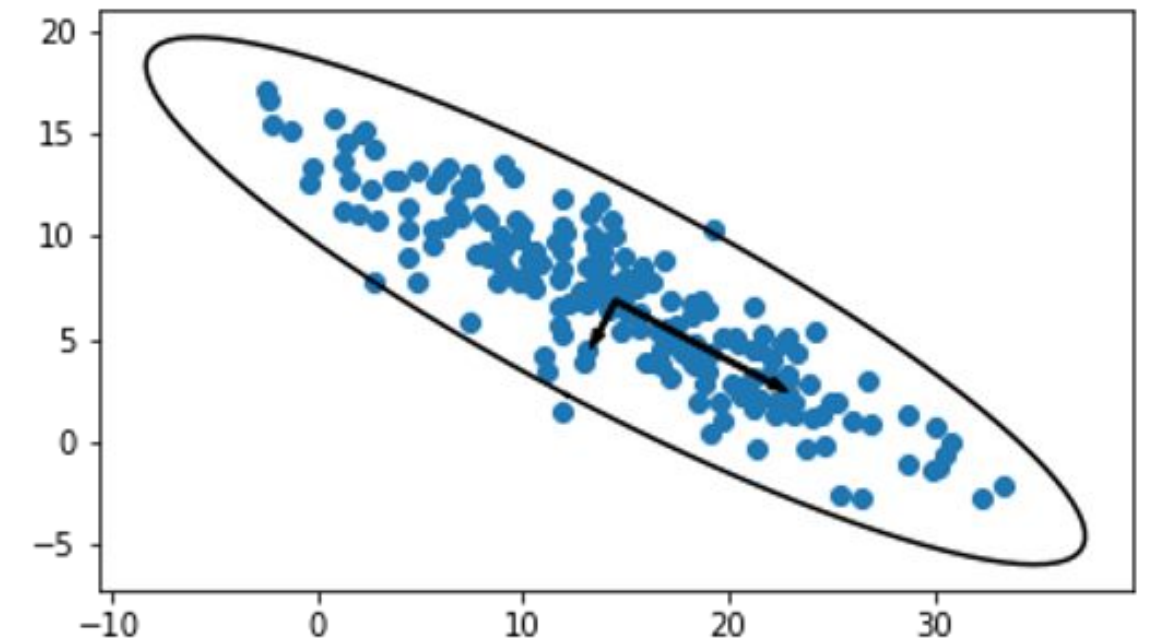
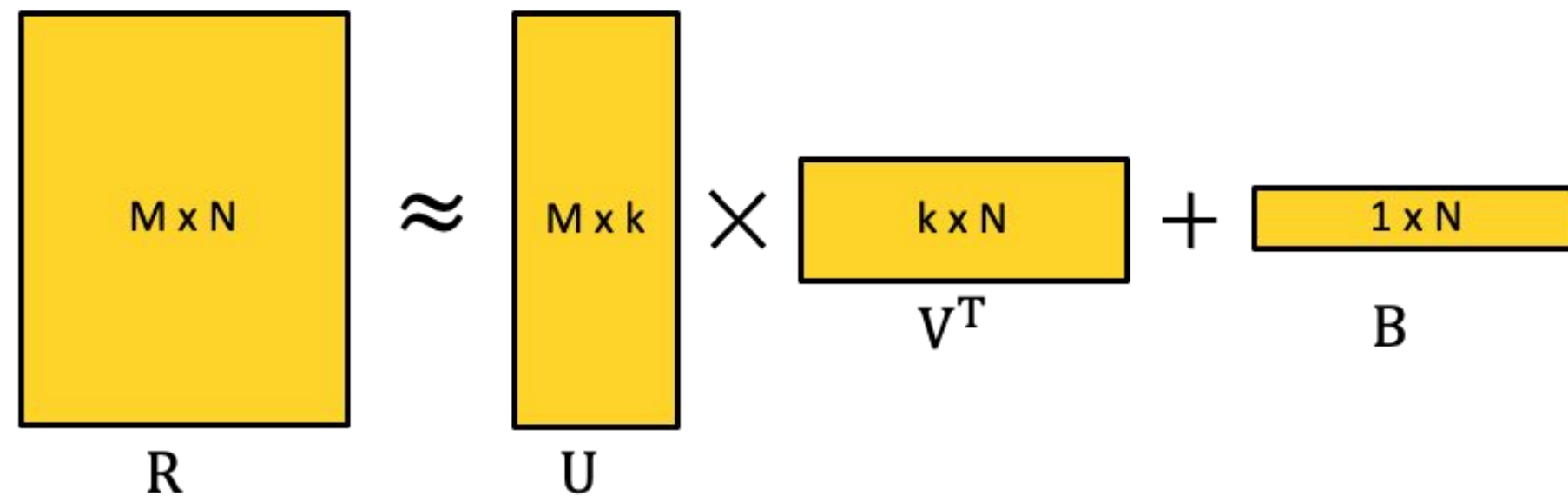
$$\sum_i \text{dist}(x_i, L_k) \rightarrow \min$$





PCA – connection with matrix decompositions

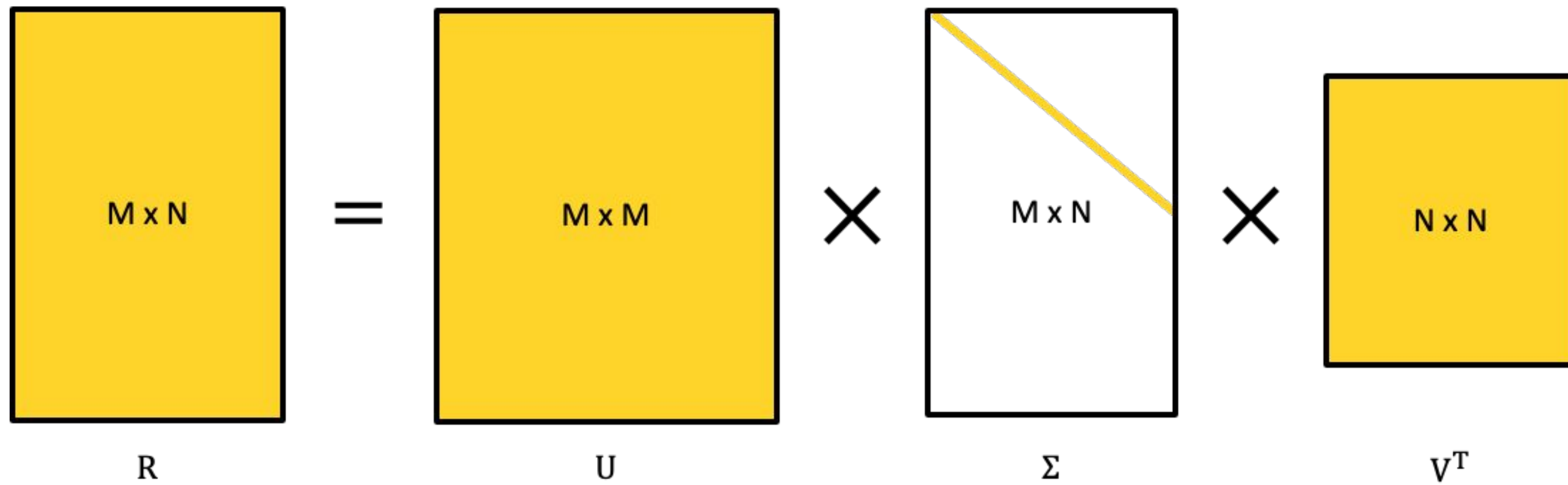
$$r_{ij} = \bar{u}_i^T \bar{v}_j + b_i$$



SVD

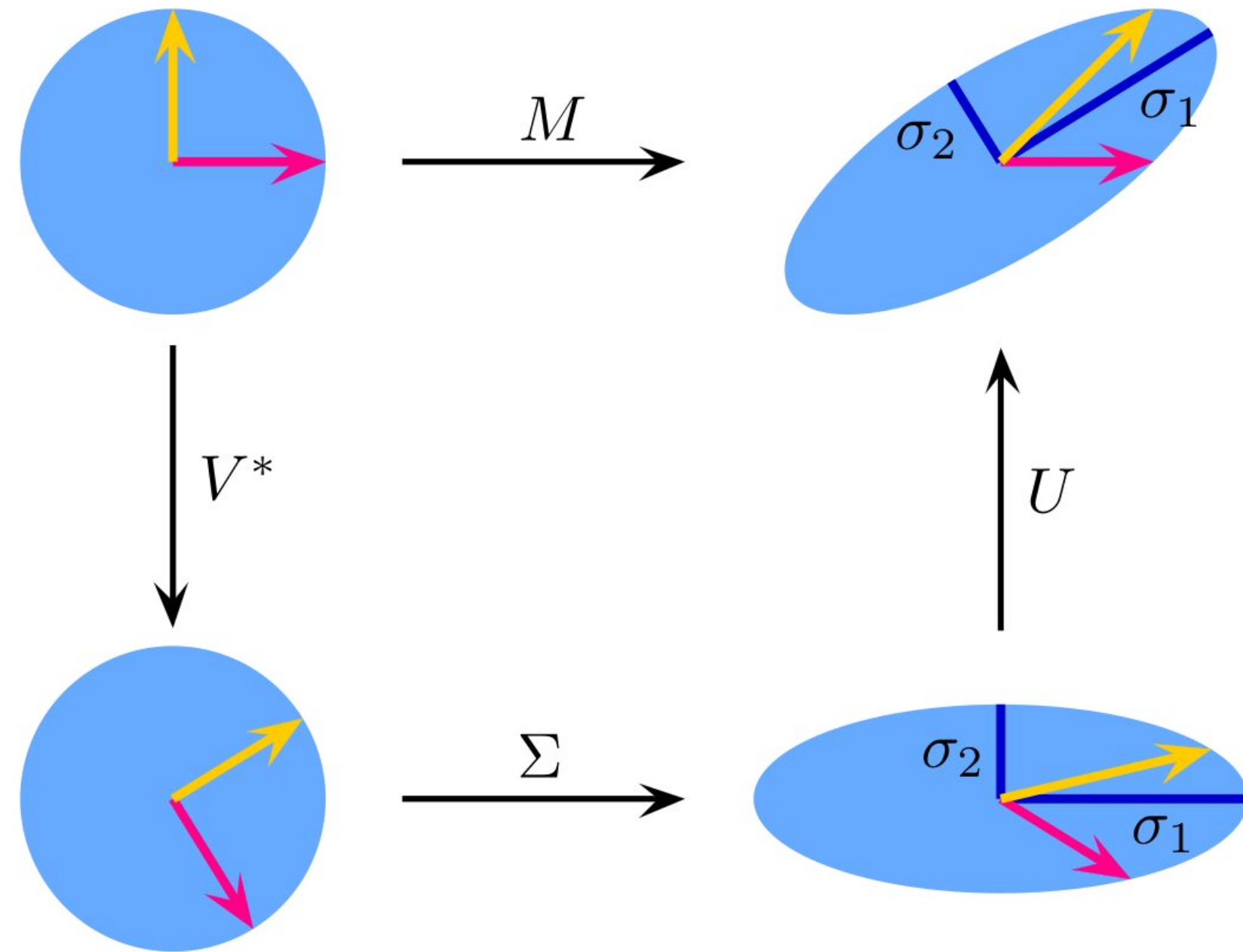


$$R = U\Sigma V^T$$
$$UU^T = U^T U = I_M$$
$$VV^T = V^T V = I_N$$



$$r_{ij} = \bar{u}_i^T \Sigma \bar{v}_j$$

SVD



$$M = U \cdot \Sigma \cdot V^*$$

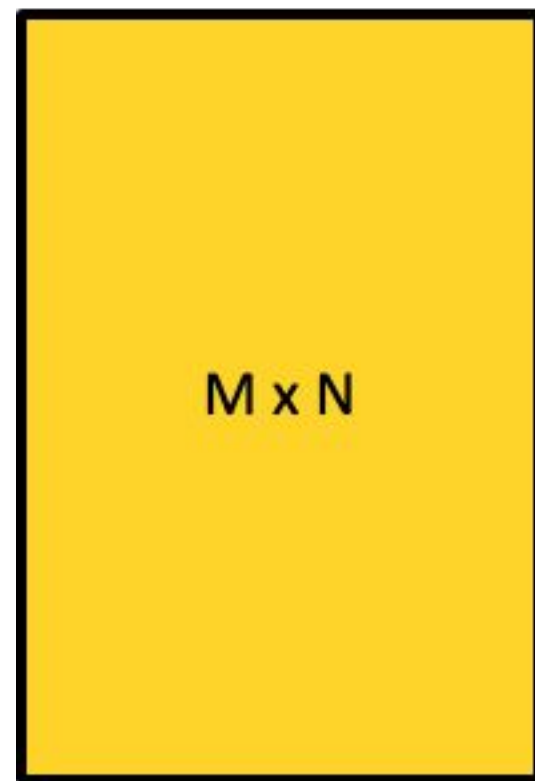


SVD compact

$$R = U\Sigma V^T$$

$$l = \text{rank}\{R\}$$

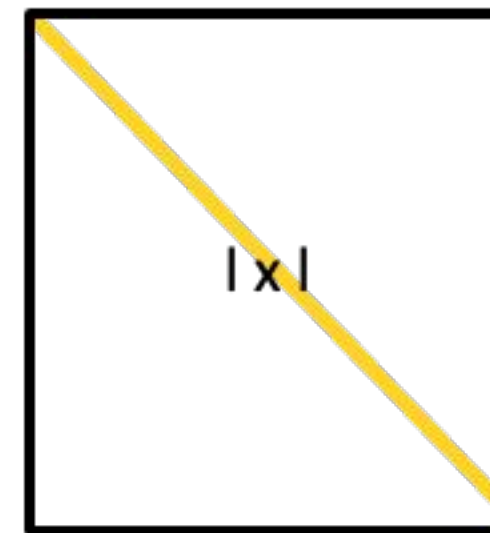
$$U^T U = V^T V = I_l$$



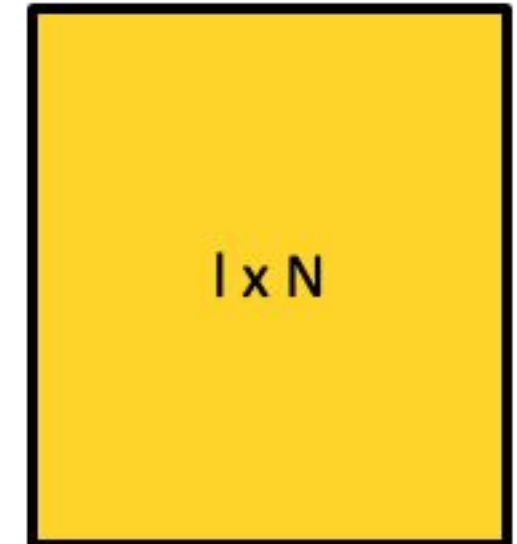
R



U



Σ



V^T

$$r_{ij} = \bar{u}_i^T \Sigma \bar{v}_j$$

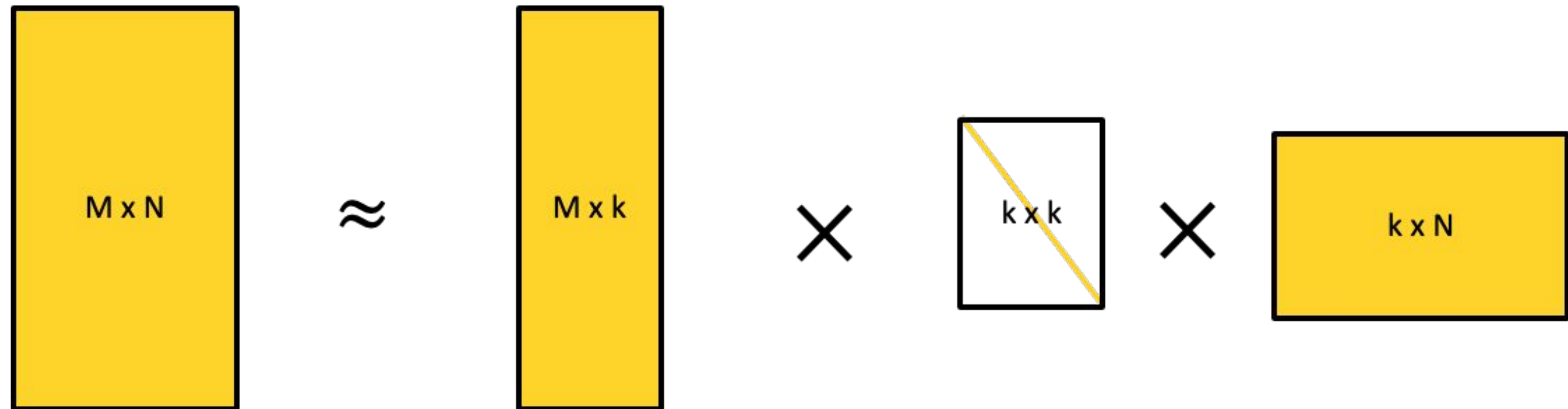


Truncated SVD

$$R \approx U \Sigma V^T$$

$$k \ll l = \text{rank}\{R\}$$

$$U^T U = V^T V = I_k$$



$$r_{ij} = \bar{u}_i^T \Sigma \bar{v}_j$$



Truncated SVD as optimization

$$R \approx U\Sigma V^T$$

$$U^T U = V^T V = I_k$$

$$U\Sigma V^T = XY^T$$

$$X^T X \neq I_k, Y^T Y \neq I_k$$

$$\min_{\substack{U, V, \Sigma \\ U^T U = V^T V = I_k \\ \sigma_{ij} = 0, i \neq j \\ \sigma_{ii} > \sigma_{jj}, i < j}} \sum_{\forall i, j} (r_{ij} - u_i^T \Sigma v_j)^2 \iff \min_{X, Y} \sum_{\forall i, j} (r_{ij} - x_i^T y_j)^2$$

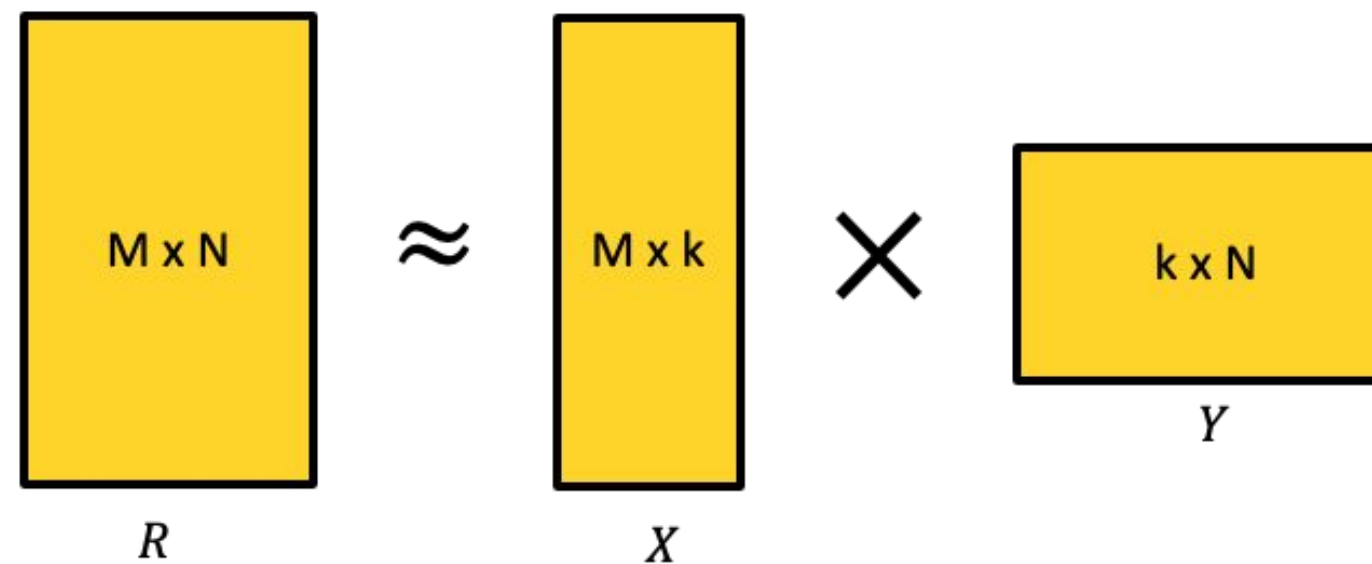
$$\min_{X, Y} \sum_{\forall i, j} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j$$

$$C_i = \frac{|\{j | r_{ij} > 0\}|^\alpha |\{i\}|}{\sum_i |\{j | r_{ij} > 0\}|^\alpha}$$



Truncated SVD as optimization

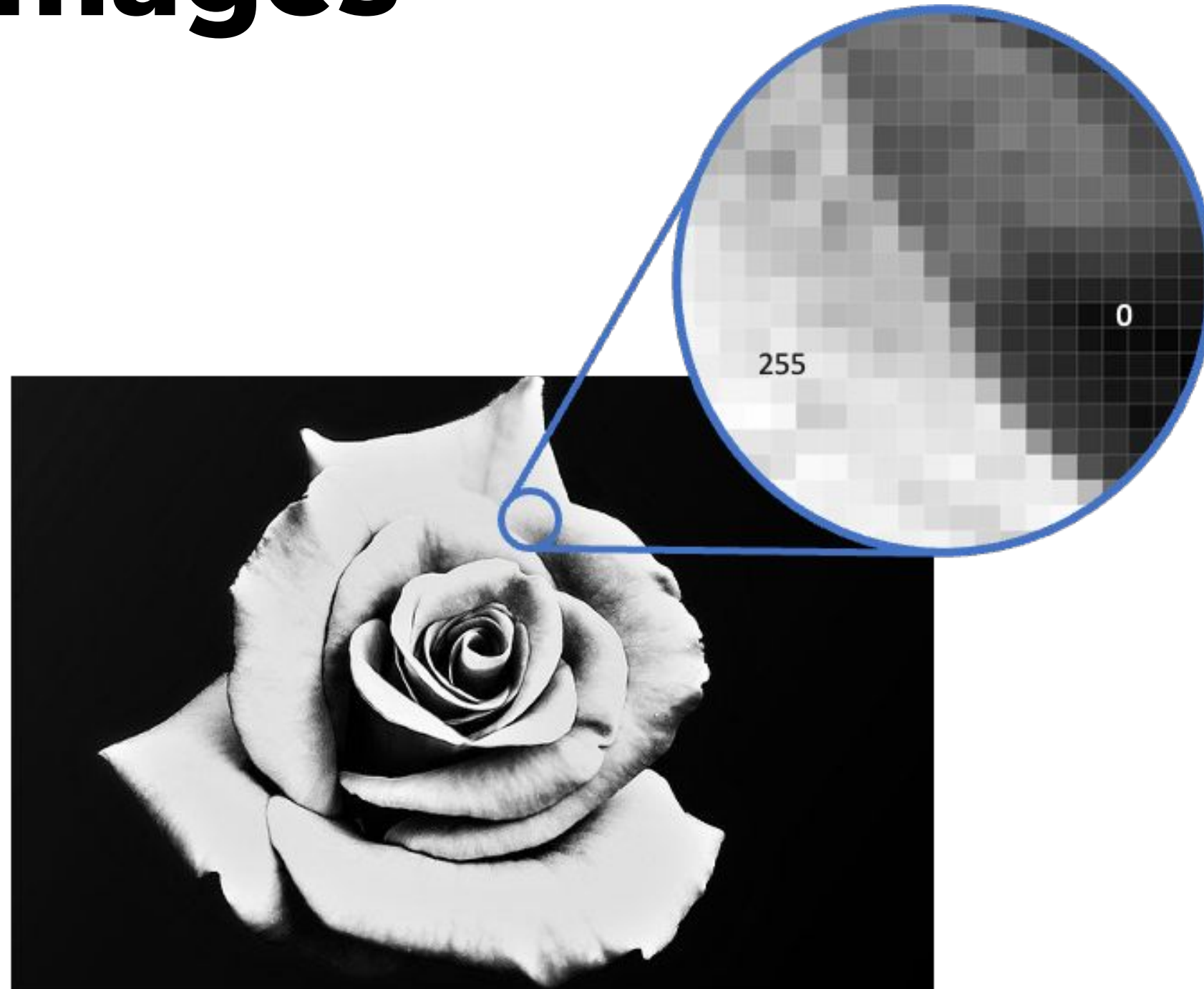
$$\min_{X,Y} \sum_{\forall i,j} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j$$



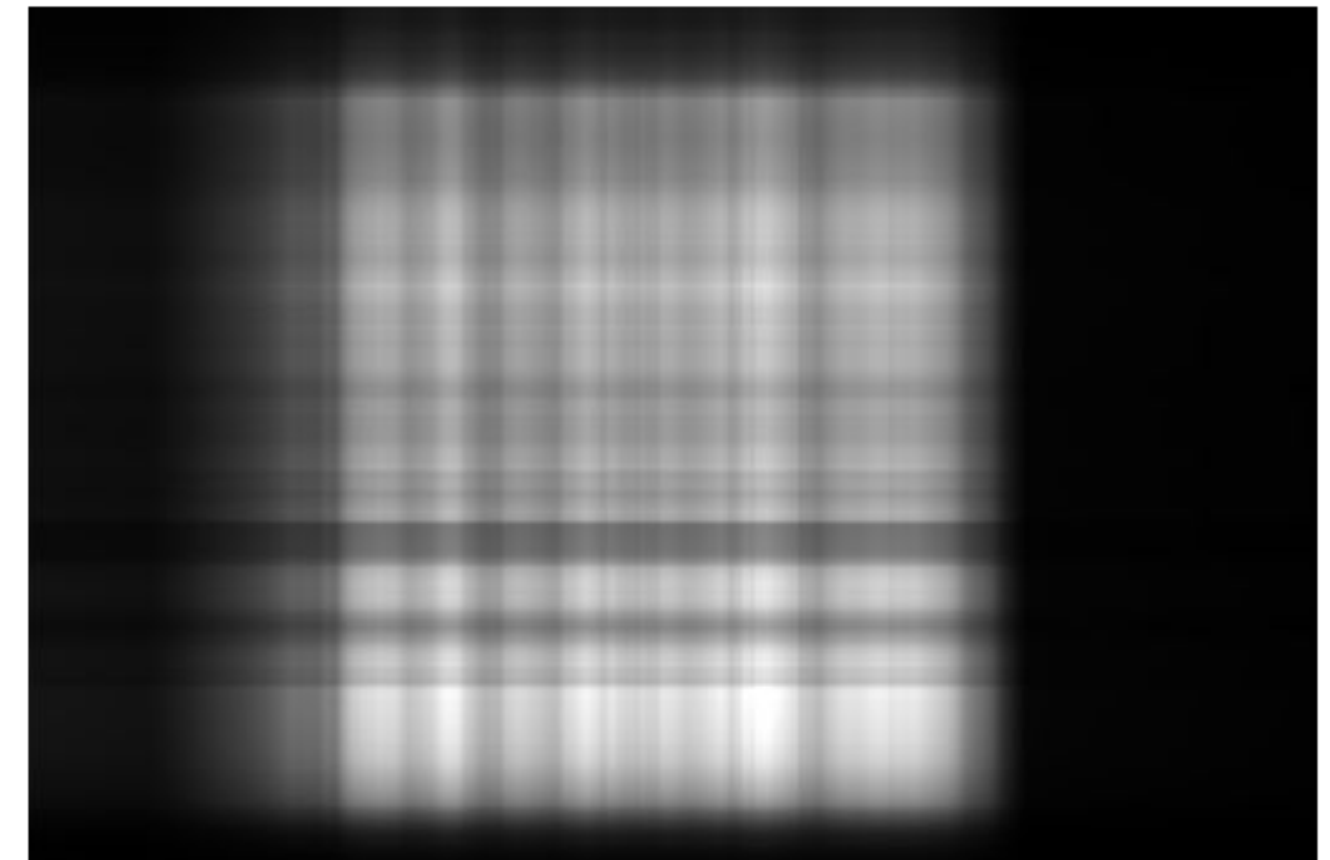
$$r_{i,j} = x_i^T y_j$$



Truncated SVD on the example of images



400 x 600 image
240 000 values



400 x 1 / 1 x 1 / 1 x 600 matrices
1001 values

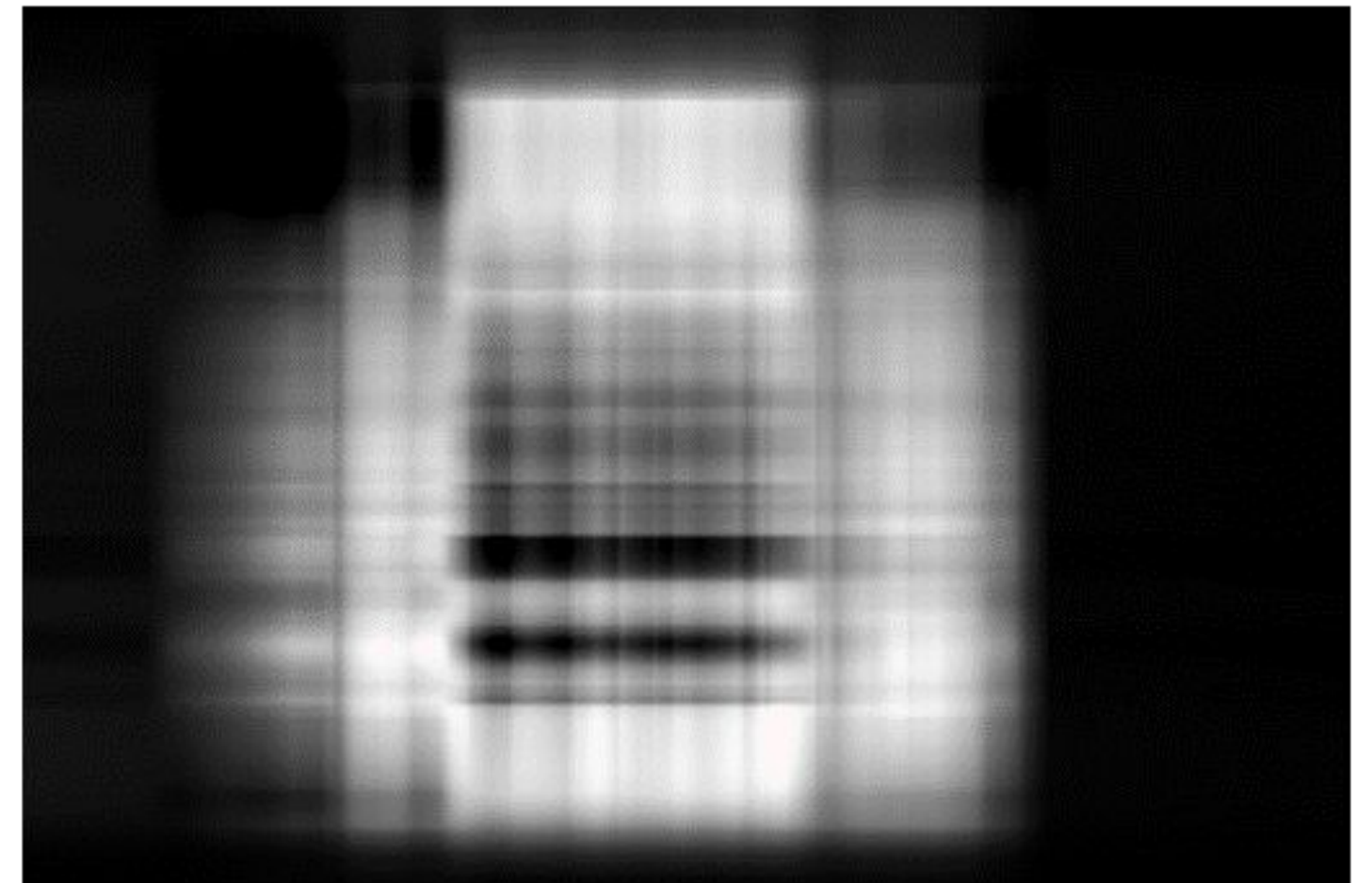


Truncated SVD on the example of images

0



400 x 600 image
240 000 values



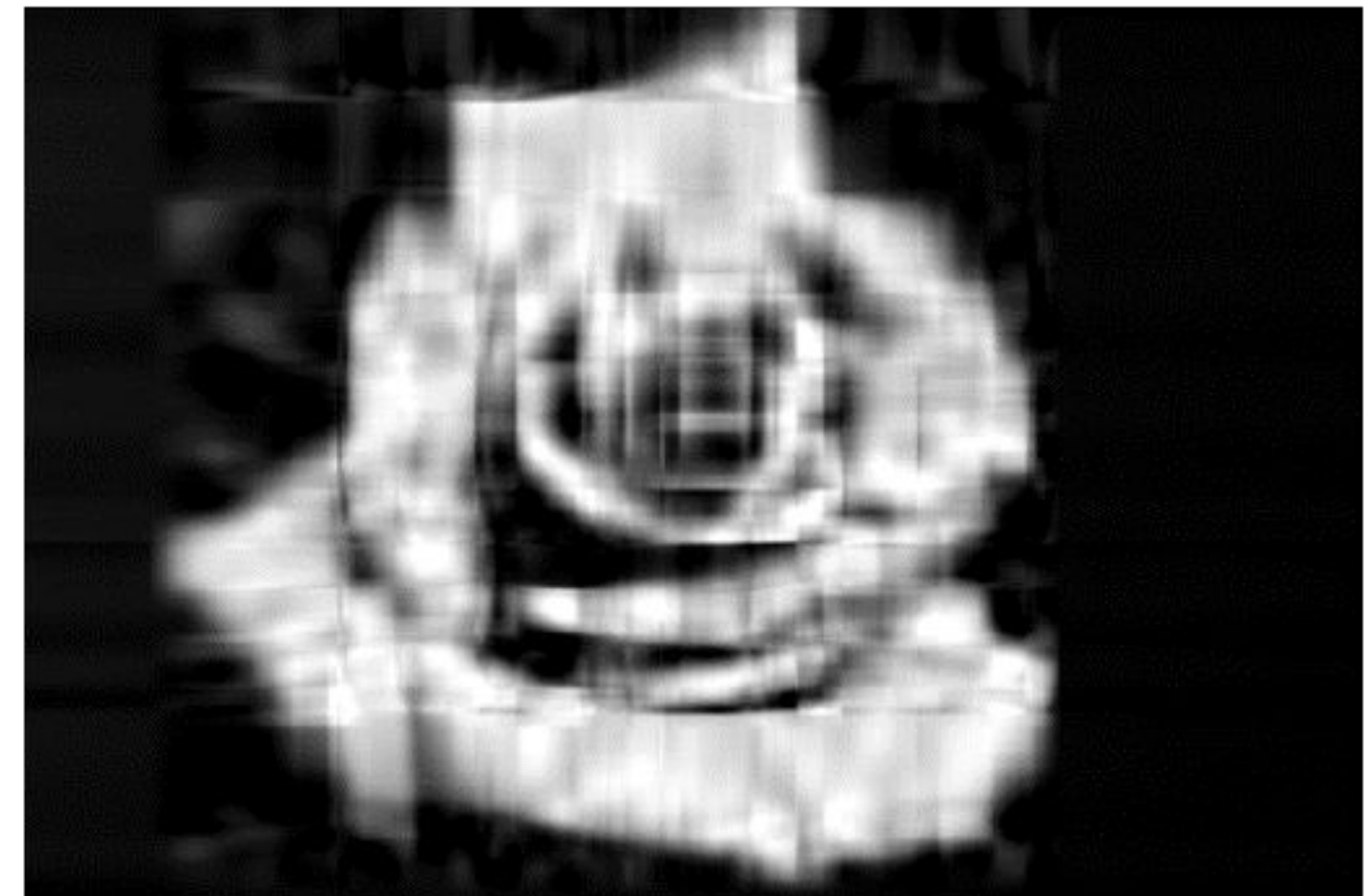
400 x 2 / 2 x 2 / 2 x 600 matrices
2002 values



Truncated SVD on the example of images



400 x 600 image
240 000 values



400 x 9 / 9 x 9 / 9 x 600 matrices
9009 values

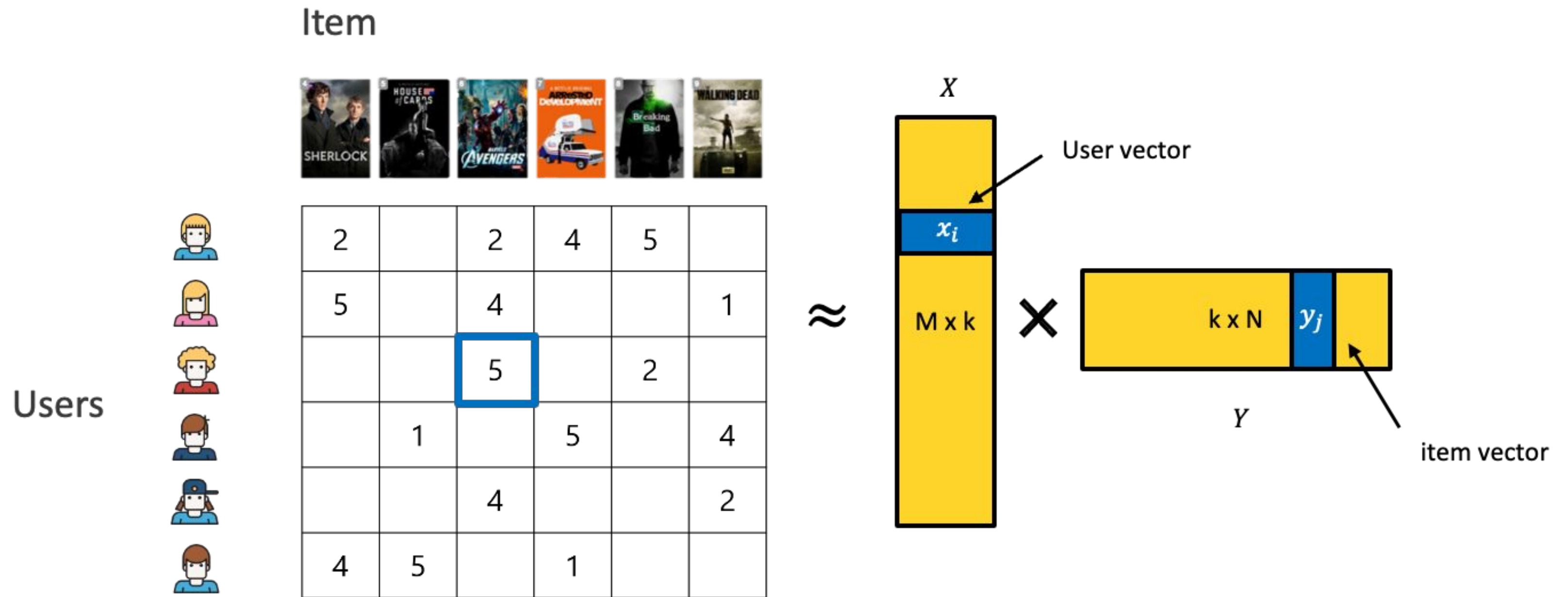
Matrix factorizations

girafe
ai

04



Funk SVD



$$\min_{X,Y} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j$$



Restored matrix



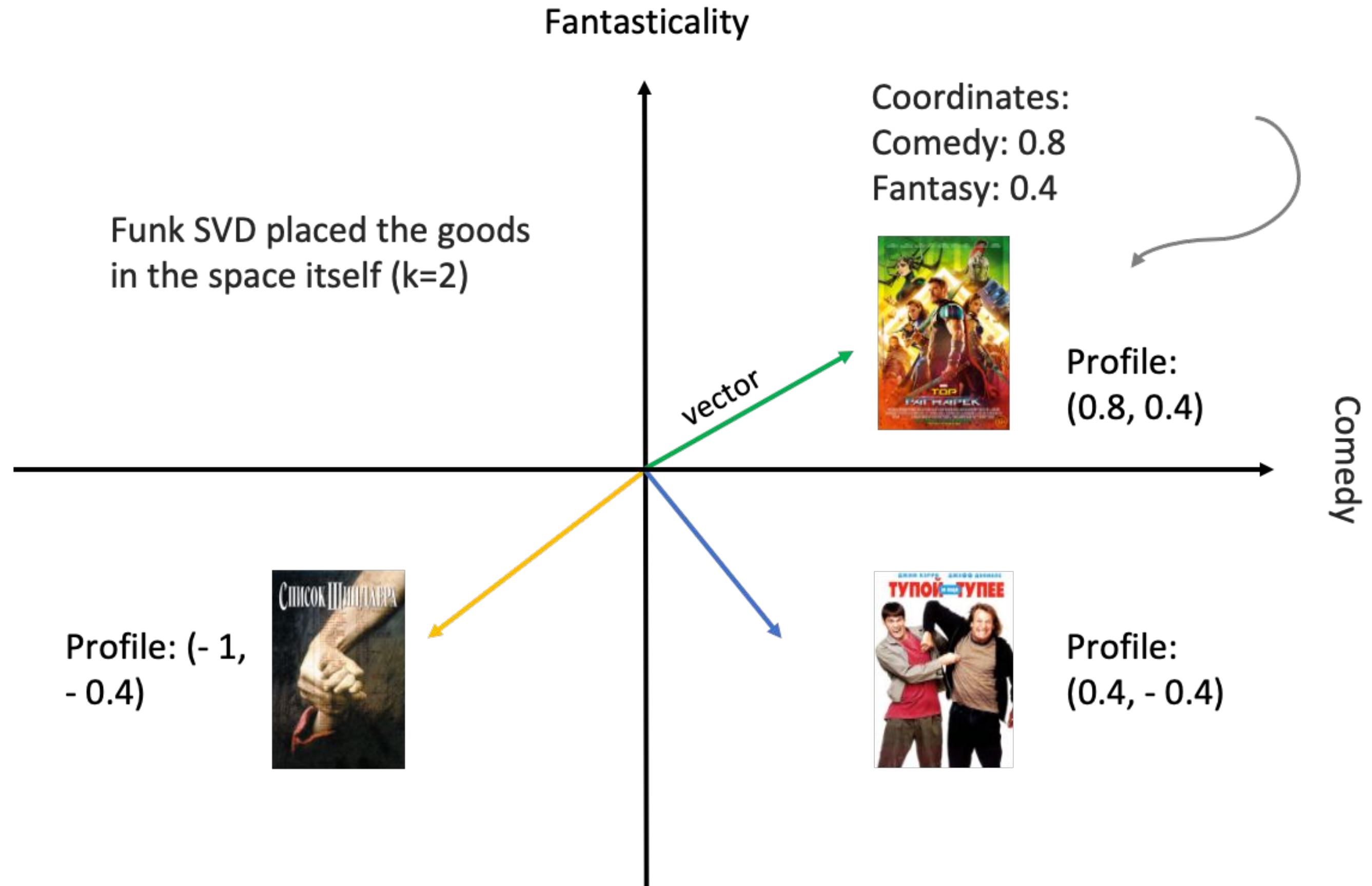
	Book 1	Book 2	Book 3	Book 4	Book 5
Person 1	✓		✓	✓	
Person 2		✓			✓
Person 3	✓	✓	✓		
Person 4				✓	✓

≈

		.9	-1	1	1	-0.9
		-0.2	-0.8	-1	0.9	1
		.88	-1.08	0.9	1.09	-0.8
		-0.9	1.0	-1.0	-1.0	0.9
		0.38	0.6	1.2	-0.7	-1.18
		-0.11	-0.9	-0.9	1.0	0.91
	1	.1				
	-1	0				
	.2	-1				
	.1	1				

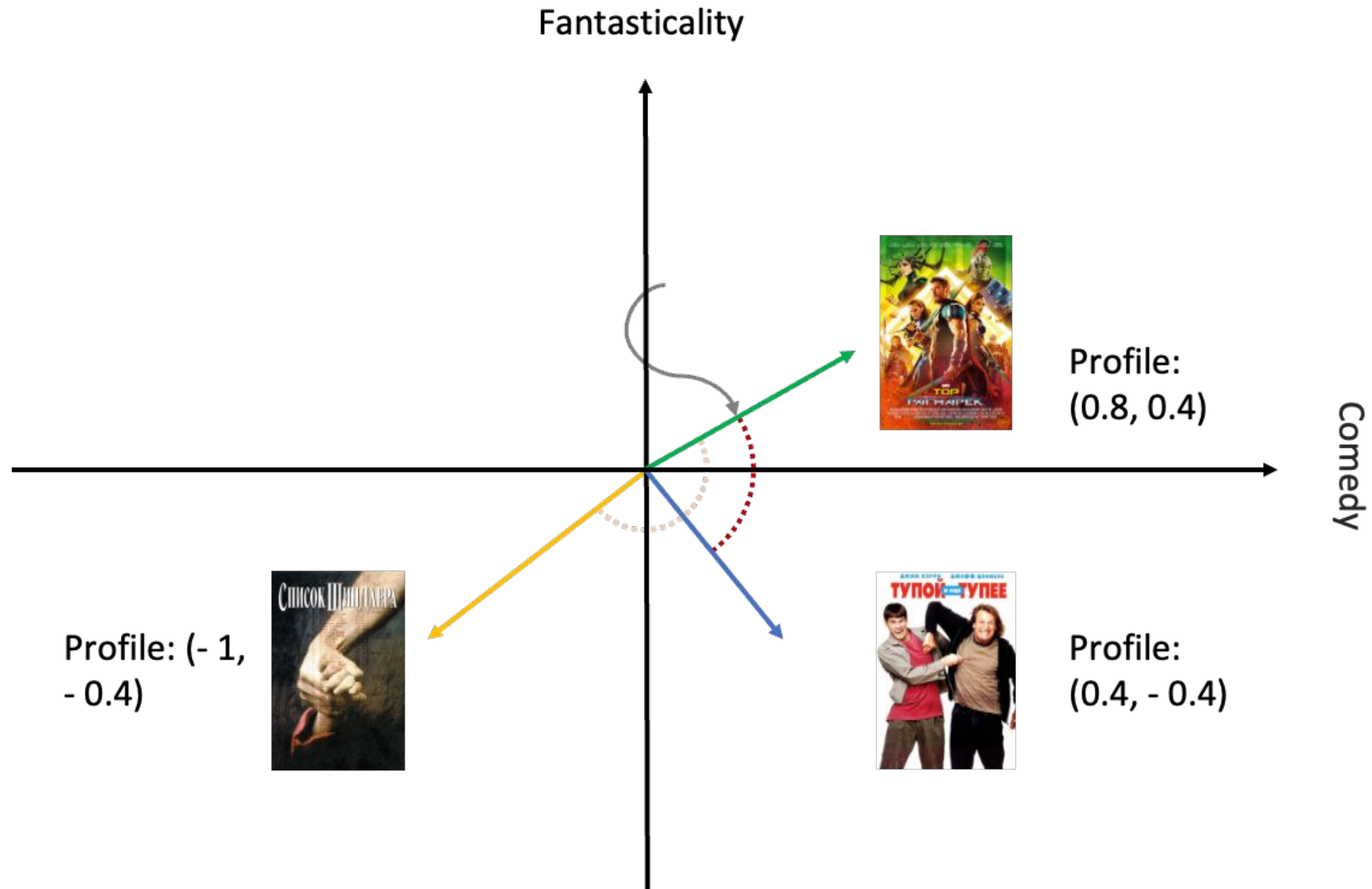


SVD Interpretation



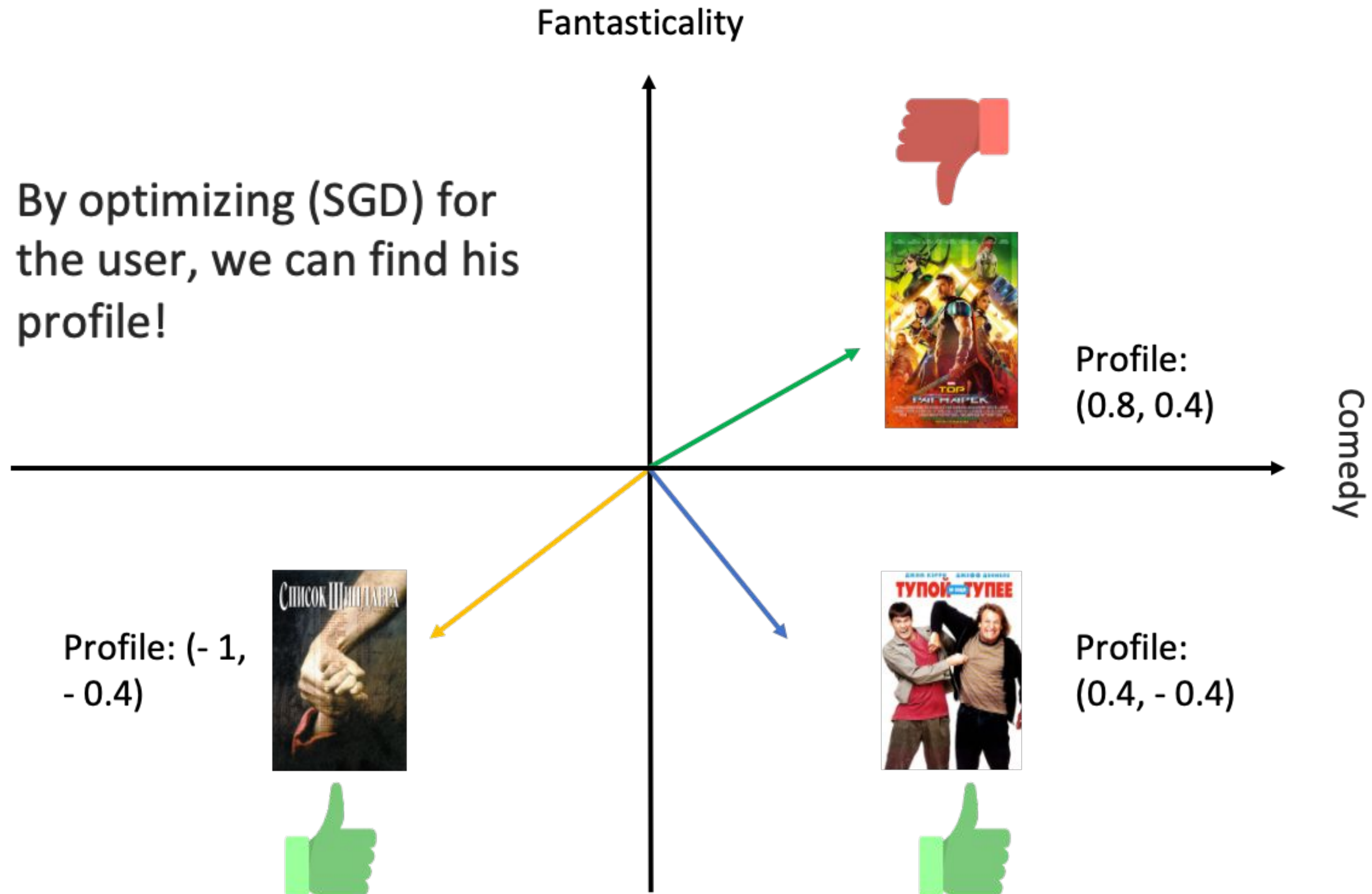


SVD Interpretation



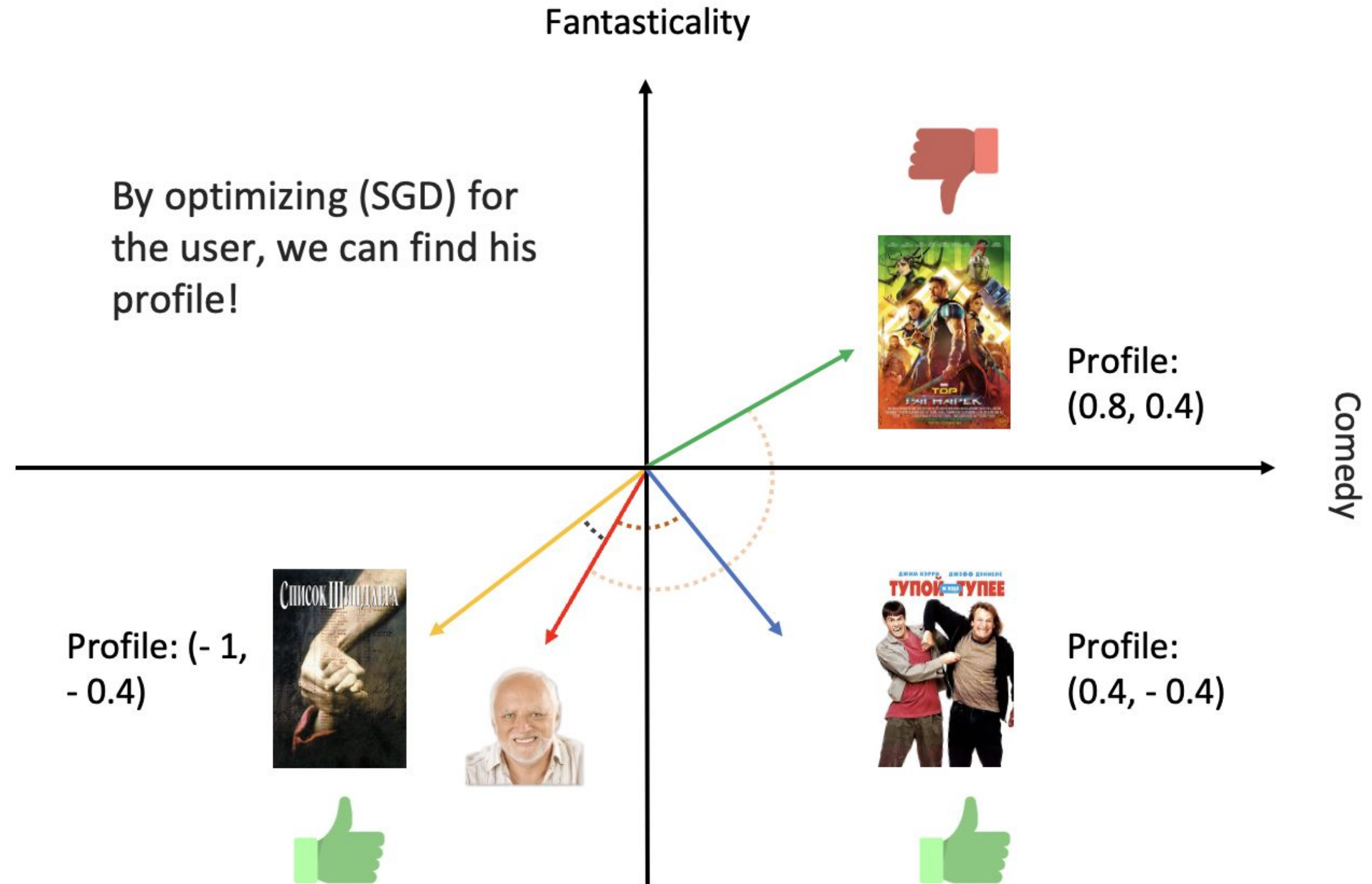


SVD Interpretation



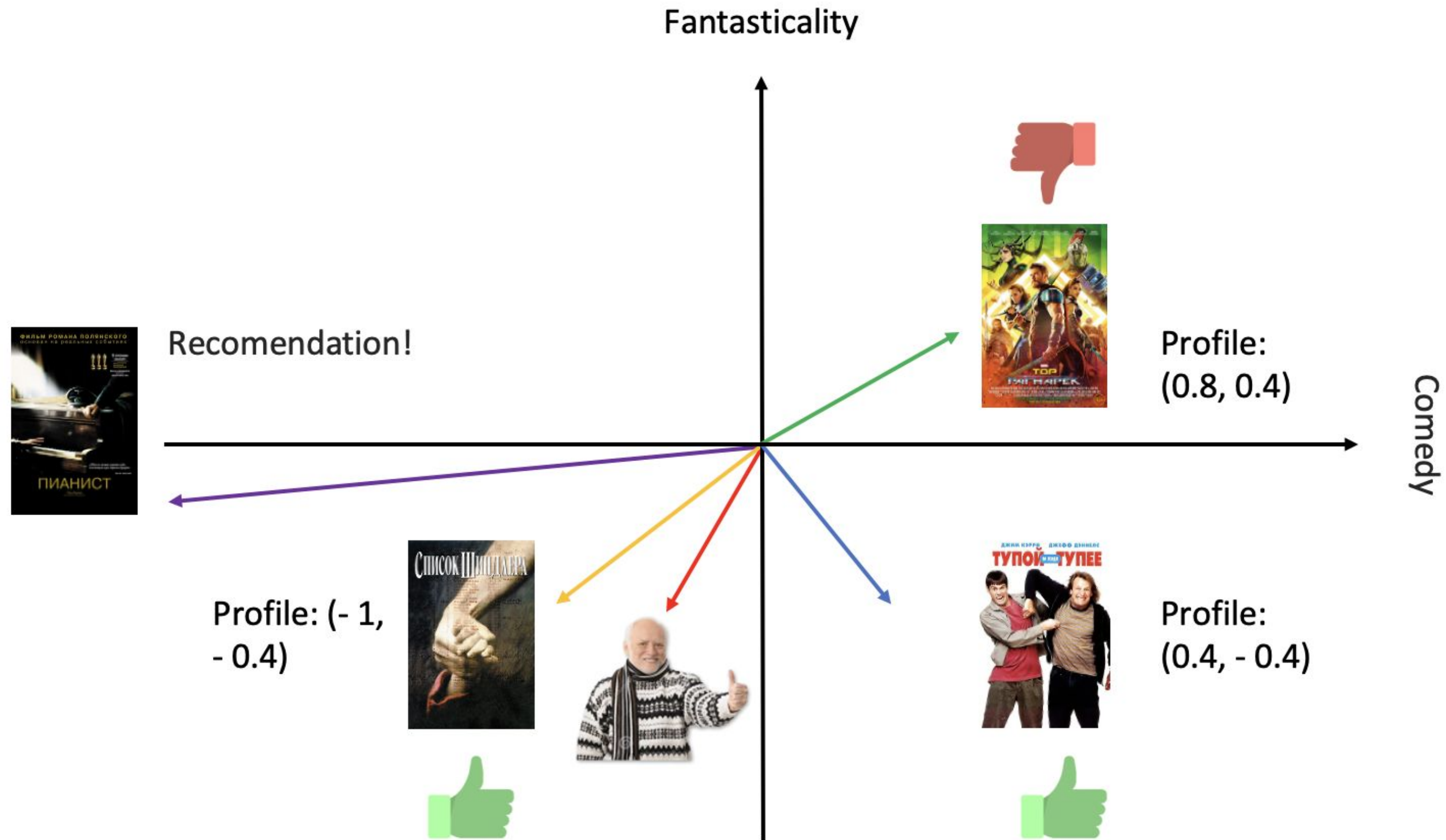


SVD Interpretation





SVD Interpretation





SVD properties

- distance is cosine
- direction stands for content
- length stands for popularity

Input: Feedback (collaboration) matrix

Trained model: Two matrices (user and item embeddings)

Prediction: for user is nearest unseen item

How many parameters are in SVD?

$$u * k + k * i = (u + i) * k \sim 2 * 10^8$$



SVD results

Matrix factorizations (MF) do not read the similarity of goods directly (as in CF).

MF attempts to describe users and products with a small set of characteristics that explain the ratings.

These characteristics may not be interpreted, at least the weights of items and users are determined with precision to the turn.

Unfortunately, SGD is difficult to parallelize.



SVD training

Straightforward way to implement it is SGD.

But it is very slow because of variables interaction.

$$\min_{X,Y} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j$$



Other MF methods

- Probabilistic Matrix Factorization (PMF)
- Bayesian Personalized Ranking (BPR)



Materials of MF

- <https://habr.com/ru/articles/486802/>
-

Alternating Least Squares

girafe
ai

05

ALS

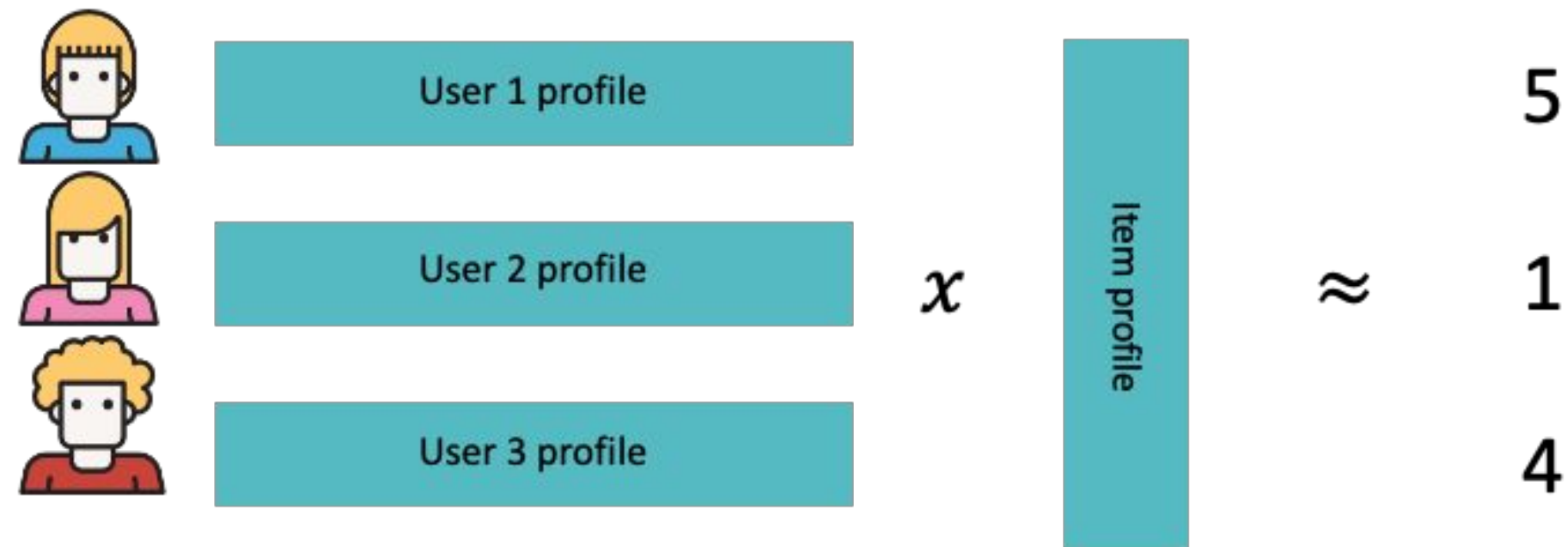


$$\min_{X,Y} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j$$

Initialize X and Y with random values.

In the loop:

- We fix the matrix X (users)
- Find the optimal matrix Y (solve the ridge regression for each product)



- And vice versa



ALS – step by x_i

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j =$$



ALS – step by x_i

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j =$$

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} r_{ij}^2 - 2 \sum_{(i,j) \in R} r_{ij} x_i^T y_j + \sum_{(i,j) \in R} (x_i^T y_j)^2 + \lambda (x_i, x_i) C_i =$$



ALS – step by x_i

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j =$$

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} r_{ij}^2 - 2 \sum_{(i,j) \in R} r_{ij} x_i^T y_j + \sum_{(i,j) \in R} (x_i^T y_j)^2 + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot x_i^T y_j + \lambda (x_i, x_i) C_i =$$



ALS – step by x_i

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j =$$

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} r_{ij}^2 - 2 \sum_{(i,j) \in R} r_{ij} x_i^T y_j + \sum_{(i,j) \in R} (x_i^T y_j)^2 + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot x_i^T y_j + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot y_j^T x_i + \lambda C_i x_i^T x_i =$$



ALS – step by x_i

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j =$$

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} r_{ij}^2 - 2 \sum_{(i,j) \in R} r_{ij} x_i^T y_j + \sum_{(i,j) \in R} (x_i^T y_j)^2 + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot x_i^T y_j + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot y_j^T x_i + \lambda C_i x_i^T x_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \left(\sum_{(i,j) \in R} r_{ij} y_j \right) + x_i^T \left(\sum_{(i,j) \in R} y_j y_j^T + \lambda C_i \right) x_i =$$



ALS – step by x_i

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j =$$

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} r_{ij}^2 - 2 \sum_{(i,j) \in R} r_{ij} x_i^T y_j + \sum_{(i,j) \in R} (x_i^T y_j)^2 + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot x_i^T y_j + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot y_j^T x_i + \lambda C_i x_i^T x_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \left(\sum_{(i,j) \in R} r_{ij} y_j \right) + x_i^T \left(\sum_{(i,j) \in R} y_j y_j^T + \lambda C_i \right) x_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T B_i + x_i^T A_i x_i = A_i^{-1} B_i$$



ALS – step by x_i

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j =$$

$$\operatorname{argmin}_{x_i} \sum_{(i,j) \in R} r_{ij}^2 - 2 \sum_{(i,j) \in R} r_{ij} x_i^T y_j + \sum_{(i,j) \in R} (x_i^T y_j)^2 + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot x_i^T y_j + \lambda (x_i, x_i) C_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \sum_{(i,j) \in R} r_{ij} y_j + \sum_{(i,j) \in R} x_i^T y_j \cdot y_j^T x_i + \lambda C_i x_i^T x_i =$$

$$\operatorname{argmin}_{x_i} -2x_i^T \left(\sum_{(i,j) \in R} r_{ij} y_j \right) + x_i^T \left(\sum_{(i,j) \in R} y_j y_j^T + \lambda C_i \right) x_i =$$

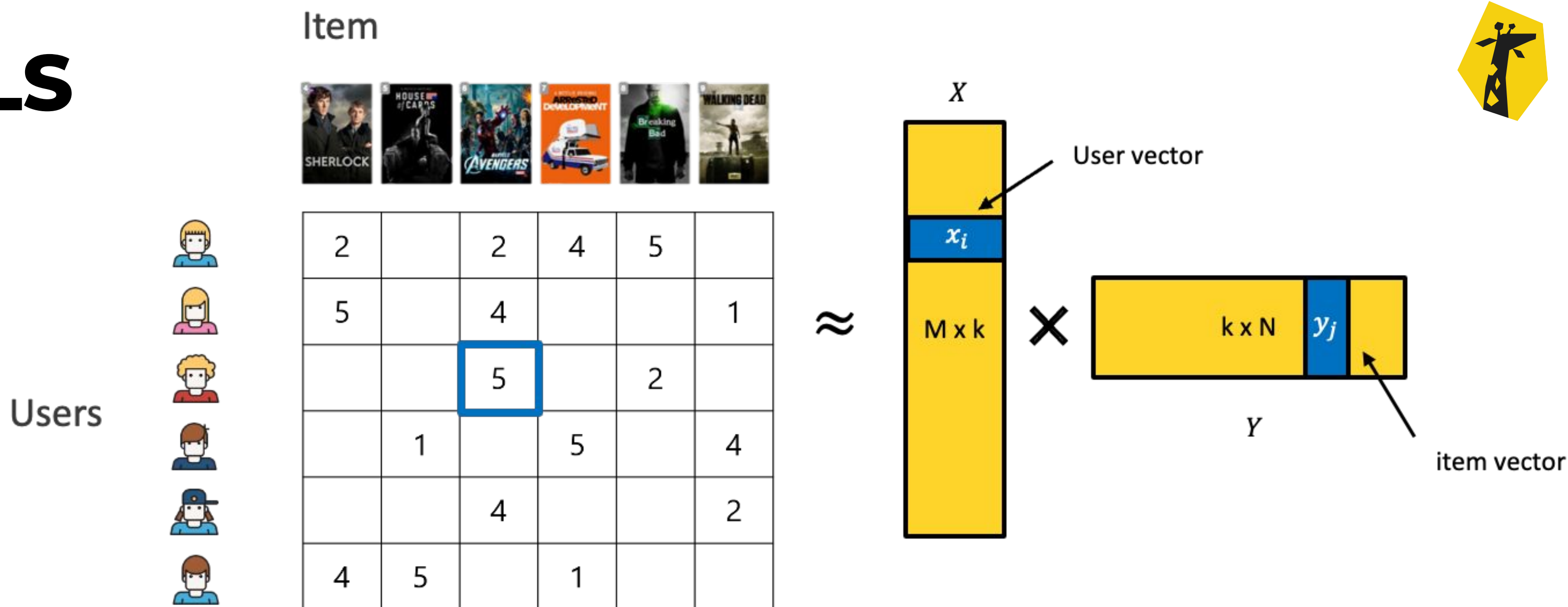
$$\left(\sum_{j|(i,j) \in R} y_j y_j^T + \lambda C_i I \right)^{-1} \left(\sum_{j|(i,j) \in R} r_{ij} y_j \right)$$

Implicit Alternating Least Squares

girafe
ai

06

IALS



$$\min_{X,Y} \sum_{(i,j) \in R} (r_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j$$

$$\min_{X,Y} \sum_{\forall i,j} c_{ij} (p_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j$$



IALS

$$p_{ij} = \begin{cases} 1, & r_{ij} > 0 \\ 0, & r_{ij} \leq 0 \text{ or } r_{ij} - \text{undefined} \end{cases}$$

Did you like it?



IALS

$$p_{ij} = \begin{cases} 1, & r_{ij} > 0 \\ 0, & r_{ij} \leq 0 \text{ or } r_{ij} - \text{undefined} \end{cases}$$

$$c_{ij} = 1 + \alpha |r_{ij}|$$

Do you like it?

How confident are you in p_{ij}



IALS

$$p_{ij} = \begin{cases} 1, & r_{ij} > 0 \\ 0, & r_{ij} \leq 0 \text{ or } r_{ij} - \text{undefined} \end{cases}$$

Do you like it?

$$c_{ij} = 1 + \alpha |r_{ij}|$$

How confident are you in p_{ij}

$$\min_{X,Y} \sum_{\forall i,j} c_{ij} (p_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j$$

$$C_i = \frac{\left(\sum_{j|(i,j) \in R} c_{ij} \right)^\alpha |\{i\}|}{\sum_i \left(\sum_{j|(i,j) \in R} c_{ij} \right)^\alpha}$$

**IALS has nothing to do with implicit
feedback!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!**





Functions of explicit and implicit

Implicit ALS is better for candidate generation

Explicit ALS is better as feature in ranking



IALS: How to optimize

$$\begin{aligned} \operatorname{argmin}_{x_i} \sum_{\forall i,j} c_{ij} (p_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j = \\ \left(\sum_{\forall j} c_{ij} y_j y_j^T + \lambda C_i I \right)^{-1} \left(\sum_{\forall j} c_{ij} p_{ij} y_j \right) = \end{aligned}$$



IALS: How to optimize

$$\begin{aligned} \operatorname{argmin}_{x_i} \sum_{\forall i,j} c_{ij} (p_{ij} - x_i^T y_j)^2 + \lambda \sum_i \|x_i\|^2 C_i + \lambda \sum_j \|y_j\|^2 C_j = \\ \left(\sum_{\forall j} c_{ij} y_j y_j^T + \lambda C_i I \right)^{-1} \left(\sum_{\forall j} c_{ij} p_{ij} y_j \right) = \\ \left(\sum_{\forall j | p_{ij}=0} c_{ij} y_j \cdot y_j^T + \sum_{\forall j | p_{ij} \neq 0} c_{ij} y_j y_j^T + \lambda C_i I \right)^{-1} \left(\sum_{\forall j | p_{ij} \neq 0} c_{ij} p_{ij} y_j \right) = \\ \left(\sum_{\forall j} y_j \cdot y_j^T - \sum_{\forall j | p_{ij} \neq 0} y_j \cdot y_j^T + \sum_{\forall j | p_{ij} \neq 0} c_{ij} y_j y_j^T + \lambda C_i I \right)^{-1} \left(\sum_{\forall j | p_{ij} \neq 0} c_{ij} p_{ij} y_j \right) = \\ \left(Y^T Y + \lambda C_i I + \sum_{\forall j | p_{ij} \neq 0} (c_{ij} - 1) y_j y_j^T \right)^{-1} \left(\sum_{\forall j | p_{ij} \neq 0} c_{ij} p_{ij} y_j \right) \end{aligned}$$



IALS – generalizations

The target p_{ij} can be not only 1 where there is a signal (we leave the implicit one equal to 0).

In confidence c_{ij} it is not necessary to use 1 as the default, any peer-to-peer matrix.three times.

The model can be slightly complicated by analogy with PCA:

$$r_{ij} \approx x_i y_j \longrightarrow r_{ij} \approx x_i y_j + b_i + b_j + \mu$$

You can combine several types of assessments with each other.



Other generalizations

ALS

$$r_{ij} \approx x_i y_j + b_i + b_j + \mu$$

SVD++

$$r_{ij} \approx \left(x_i + \frac{1}{\sqrt{|\{s | p_{is} \neq 0\}|}} \sum_{\forall s | p_{is} \neq 0} \widehat{y}_s \right) y_j + b_i + b_j + \mu$$

timeSVD++

$$r_{ij}(t) \approx \left(x_i(t) + \frac{1}{\sqrt{|\{s | p_{is} \neq 0\}|}} \sum_{\forall s | p_{is} \neq 0} \widehat{y}_s \right) y_j + b_i(t) + b_j(t) + \mu$$

Various targets

girafe
ai

07



Target can be anything

It can be

- likes
- rating
- time spent
- some custom formula

Different targets correspond to different dependencies



Several ALS at time

Functions of embeddings:

- candidate generators
- features in ranking
- subscription recommendation



Cross-product data

Weak in most cases



User vectors from content item embeddings

We take item embeddings from any content NN and perform one step of ALS to get user embeddings



Data drift

Over time data changes:

- user preferences
- item set

Solution: retrain model on new data

Issues:

- compute expensive
- predictions will change
 - almost not changing



Is it important?

Yes, it is!

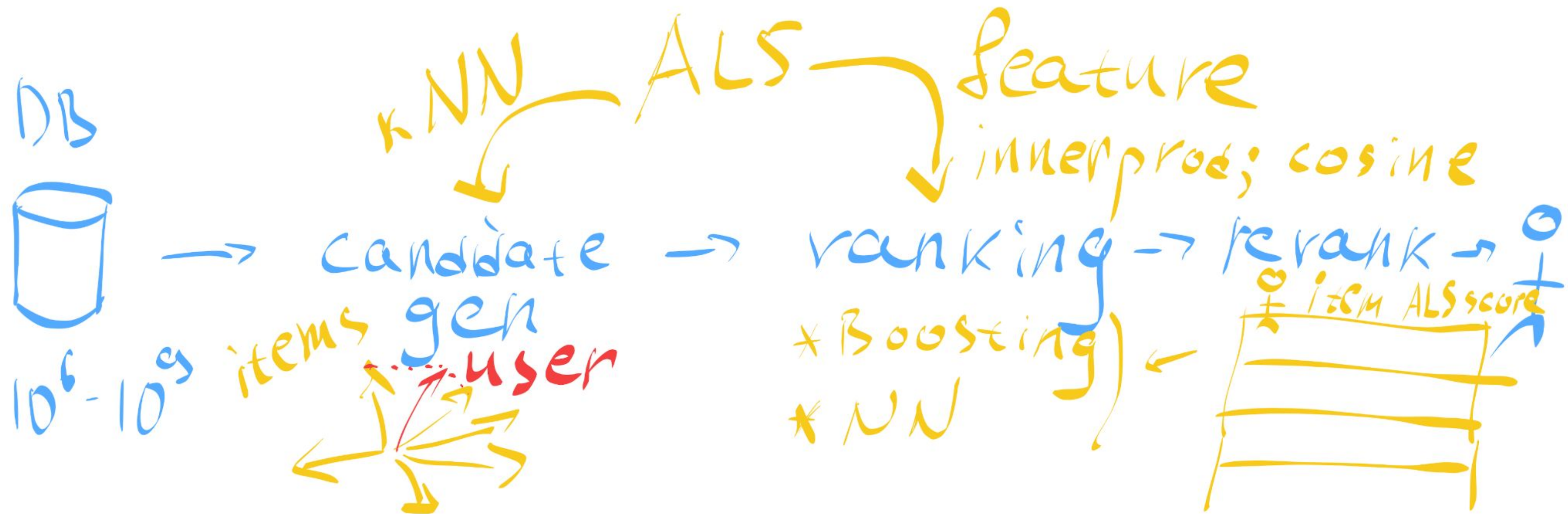
Typical latency of recommendations in popular services are:

- Youtube - less than 1 minute (15 seconds subjectively)
- TikTok - less than 1 minute (5 seconds subjectively)
-



ALS usage scenario

- as candidate generator (usually iALS)
- as feature to ranking (usually eALS)



Sparse matrices

girafe
ai

08



Types of sparse matrices

<https://docs.scipy.org/doc/scipy/reference/sparse.html>

- Coordinate List (COO)
- Compressed Sparse Row (CSR)
- Compressed Sparse Column (CSC)
- Dictionary of Keys (DOK)
- Block Sparse or Diagonal Formats (BSR, DIA)

Incremental training

girafe
ai

09

Online and offline recommendations



For a long time (till 2015) these two approaches competed

- offline
 - train huge model
 - predict offline
 - return cached result
- online
 - models are smaller
 - infer in real time

Try not to use caches at all. See VK on HighLoad 2023

Realtime fine tuning



We want our model to update fast, so we update matrices separately



Other generalizations (at the next lecture)

- SLIM
- Factorization machine
- DSSM
- Other losses

Content moderation



- Could be based on popularity bias
- Basically give place for all
- After reaching



ALS implementations

- <https://github.com/lyst/lightfm>
- <https://github.com/benfred/implicit/>
- <https://github.com/NicolasHug/Surprise>
- <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.recommendation.ALS.html>
 - <https://spark.apache.org/docs/latest/ml-collaborative-filtering.html>
- <https://recbole.io/> <https://github.com/RUCAIBox/RecBole>
- <https://github.com/MobileTeleSystems/RecTools>



Full pipeline of IR

<https://github.com/metarank/metarank> - full system implementation

<https://github.com/twitter/the-algorithm> - Twitter recommender system

Thanks for attention!

Questions?



girafe
ai

